

Hackathon Web Transactionnelle

Plateforme de Gestion de Ligues Sportives Communautaires

Checkpoint de progression – Séance 08h00 à 11h00

Durée maximale du hackathon : 48 heures

Objectif de cette séance

Vous êtes déjà en plein hackathon. Cette séance n'est donc pas un cours magistral classique. Elle sert à vérifier votre avancement, clarifier votre MVP, débloquer les problèmes techniques et vous pousser vers une version fonctionnelle du projet.

Votre objectif n'est pas de tout faire. Votre objectif est de livrer une version utilisable, cohérente, testable et présentable de la plateforme.

Rappel du sujet

Vous devez développer une plateforme web moderne permettant de gérer des ligues sportives communautaires. L'application connecte principalement deux types d'utilisateurs :

- des **organiseurs**, qui créent et gèrent des tournois, des équipes, des demandes et des matchs ;
- des **joueurs**, qui recherchent des équipes, envoient des demandes d'adhésion et suivent leurs matchs.

L'idée générale est de créer une sorte de plateforme de rencontre sportive communautaire, comparable à un « Meetup » pour les sports amateurs.

1. Informations générales de l'équipe

Nom du projet	Tournament Sports
Nom de l'équipe	DNA
Membres de l'équipe	Dominique Lemay et Alexandre Leblanc
Répartition des rôles	Frontend : Dom Backend : Alex et Dom BDD / Prisma : Alex Auth / Stripe : Dom
Lien GitHub	https://github.com/domlemay/TournamentSports .git

2. Stand-up rapide de début de séance

Chaque équipe doit répondre rapidement aux questions suivantes.

Question	Réponse de l'équipe
Qu'avez-vous fait hier entre 14h et 17h ?	Lancer le projet avec toute les dépendance créer les lien clerk et neon créer l'architecture complet et créer les page de — profile — Match — Auth via Clerk — Equipe — Demande d'adhésion
Quelle partie du projet est déjà commencée ?	On commence ce matin les Paiements stripe
Qu'est-ce qui bloque actuellement l'équipe ?	Honnêtement rien (on trouve les solutions a fur et mesure)
Quelle fonctionnalité voulez-vous rendre visible avant 11h ?	Les Paiements

Règle du hackathon

Une fonctionnalité non visible, non testable ou non démontrable ne compte pas vraiment. À partir de maintenant, vous devez penser en termes de démonstration : qu'est-ce que vous pouvez réellement montrer ?

3. MVP obligatoire : checklist de progression

Cochez ce qui est déjà fait, puis indiquez ce qui reste à faire.

Fonctionnalité MVP	Statut	Notes / Problèmes / Prochaines étapes
Authentification avec Clerk	☒	simple, commence a être habituer
Synchronisation User Clerk vers Prisma	☒	quelque erreur dans le shema.prisma fournis mais vite corrigé

Gestion des rôles : PLAYER, ORGANIZER, ADMIN	☒	Pas rencontré vraiment de problématique
Profil joueur : ville, sport, niveau, poste	☒	Page quand même assez simple a monter
Dashboard organisateur	☒	La page en tant que telle est simple
Création et gestion des tournois	☒	Créer naturellement avec le Dashboard Or- ganisateur
Création et gestion des équipes	☒	Un peu plus long a monter
Recherche d'équipes côté joueur	☒	un peu plus de difficulté (peut-être du a la fatigue)
Demandes d'adhésion	☒	aucun problème rencontré
Acceptation / refus des de- mandes	☒	Terminer durant le cour
Matches : création, liste, mo- dification simple	☒	Terminer durant le cour
Paieement Stripe pour tour- noi payant	☒	quelque bug a regler
Section admin custom	☒	completé

4. Priorité absolue : la fonctionnalité centrale

Fonctionnalité centrale du projet

La fonctionnalité la plus importante du sujet est le système de **demandes d'adhésion**. C'est elle qui donne une vraie logique métier à l'application.

Le scénario principal est le suivant :

1. Un organisateur crée un tournoi.
2. L'organisateur crée une ou plusieurs équipes dans ce tournoi.
3. Un joueur recherche une équipe disponible.
4. Le joueur envoie une demande d'adhésion.
5. Si le tournoi est payant, le joueur doit passer par Stripe.
6. L'organisateur voit les demandes reçues.
7. L'organisateur accepte ou refuse la demande.
8. Si la demande est acceptée, le joueur devient membre de l'équipe.

Votre état actuel sur cette fonctionnalité

Question	Réponse
Avez-vous déjà créé le modèle JoinRequest dans Prisma ?	Oui
Avez-vous empêché les demandes en double ?	Oui
Avez-vous vérifié la capacité maximale d'une équipe ?	Oui
Avez-vous une Server Action createJoinRequest() ?	Oui
Avez-vous une Server Action acceptJoinRequest() ?	Oui
L'acceptation se fait-elle dans une transaction Prisma ?	Oui
Le joueur voit-il le statut de ses demandes ?	Oui

L'organisateur voit-il les demandes reçues ?	Oui
--	-----

5. Modèle de données : vérification rapide

Votre base de données doit au minimum contenir les éléments suivants.

- ☒ User
- ☒ PlayerProfile
- ☒ Tournament
- ☒ Team
- ☒ JoinRequest
- ☒ Match

État du schéma Prisma

Modèle	Créé ?	Notes
User	Oui / Non / Partiel	Oui
PlayerProfile	Oui / Non / Partiel	Oui
Tournament	Oui / Non / Partiel	Oui
Team	Oui / Non / Partiel	Oui
JoinRequest	Oui / Non / Partiel	Oui
Match	Oui / Non / Partiel	Oui

6. Pages et routes à viser

Page / Route	Rôle	Statut
/	Landing publique	/src/app/page.tsx

/sign-in et /sign-up	Authentification Clerk	/src/app/(auth)/sign-in*out/[...sign-in*out]/page.tsx
/profile	Profil joueur	/src/app/(player)/profile/page.tsx
/dashboard	Dashboard organisateur	/src/app/(organizer)/dashboard/page.tsx
/tournaments	Liste / gestion des tournois	/src/app/(organizer)/tournaments/page.tsx
/tournaments/[id]	Détail d'un tournoi	/src/app/(organizer)/tournaments/[ID]/page.tsx
/teams	Recherche d'équipes	/src/app/(player)/teams/page.tsx
/teams/[id]	Détail d'une équipe	/src/app/(player)/teams/[id]/page.tsx
/my-requests	Demandes du joueur	/src/app/(player)/my-request/page.tsx
/requests	Demandes reçues par l'organisateur	/src/app/(organizer)/requests/page.tsx
/matches	Matches	/src/app/matches.ts
/admin	Administration custom	/src/app/admin/page.tsx

7. Server Actions et Route Handlers

Server Actions prioritaires

- ☒ updatePlayerProfile()
- ☒ createTournament()
- ☒ updateTournament(id)
- ☒ deleteTournament(id)
- ☒ createTeam()
- ☒ createJoinRequest()
- ☒ cancelJoinRequest(id)
- ☒ acceptJoinRequest(id)

- ☒ rejectJoinRequest(id)
- ☒ createMatch()
- ☒ updateMatchScore(id)

Route Handlers nécessaires

- ☒ POST /api/webhooks/clerk
- ☒ POST /api/webhooks/stripe
- ☒ POST /api/checkout

8. Stripe : décision immédiate

Attention

Si votre application permet de créer un tournoi avec des frais d'inscription supérieurs à 0, le paiement Stripe devient obligatoire dans le MVP.

Si vous êtes en retard techniquement, commencez d'abord par faire fonctionner tout le parcours avec un tournoi gratuit. Ensuite seulement, ajoutez le flux Stripe pour les tournois payants.

Question	Réponse
Avez-vous prévu des tournois payants ?	Oui
Le champ entryFee existe-t-il dans Tournament ?	Oui
Le champ paymentStatus existe-t-il dans JoinRequest ?	Oui
La route /api/checkout est-elle commencée ?	Oui
Le webhook Stripe est-il commencé ?	Oui
Avez-vous testé Stripe CLI ?	Oui

9. Objectif concret avant 11h00

Chaque équipe doit choisir une seule priorité principale pour la fin de la séance.

À 11h00, votre équipe doit pouvoir montrer au moins une chose concrète :

- une page fonctionnelle ;
- une connexion réussie ;
- une création de tournoi ;
- une création d'équipe ;
- une demande d'adhésion ;
- une acceptation de demande ;
- une liste affichée depuis Prisma ;
- ou une autre fonctionnalité visible et testable.

Élément	Réponse de l'équipe
Fonctionnalité à démontrer avant 11h00	Possible
Responsable principal de cette fonctionnalité	Dom
Ce qui est déjà fait	Presque tout
Ce qui reste à faire	Tester
Risque principal	de s'ennuyer d'Eddine après les cours
Plan B si cela ne fonctionne pas	Rester en contact avec Eddine

10. Priorisation : Must Have, Nice to Have, Abandon

Must Have

Ce qui doit absolument être fait pour que le projet soit présentable.

- compléter le projet
- tester le projet

Nice to Have

Ce qui est intéressant seulement si le MVP fonctionne déjà.

- Ajouter une embellissement du frontend
- Intégrer un forum
- intégrer une messagerie

À abandonner si le temps manque

Ce qui peut être retiré sans détruire la logique principale du projet.

- le superflux

11. Pièges techniques à éviter

- Ne pas importer Prisma dans un Client Component.
- Ne pas oublier revalidatePath après une mutation.
- Ne jamais faire confiance aux données envoyées par le client.
- Toujours valider les entrées côté serveur avec Zod.
- Ne jamais exposer les secrets dans NEXT_PUBLIC_*.
- Ne pas vérifier l'auth uniquement dans le middleware : vérifier aussi dans les Server Actions sensibles.
- Ne pas passer trop de temps sur le design avant d'avoir une logique fonctionnelle.
- Ne pas commencer les bonus avant d'avoir terminé le MVP.

12. Mini pitch technique

À la fin de la séance, chaque équipe doit être capable de présenter rapidement son avancement.

Notre projet s'appelle : Tournoiement Sports

Notre plateforme permet de : S'inscrire en tant que Player ou Organizer et peut être les deux à la fois, Rejoindre une équipe, créer des match, participer à des tournois payant ou pas.

Notre utilisateur joueur peut : Rejoindre une équipe, Participer à des matches et des tournois.

Notre utilisateur organisateur peut : créer des équipe créer des tournois, créer des matches et les organiser

Notre fonctionnalité centrale est : plateforme pour les joueurs pour gérer leurs match et tournois.

Notre base de données contient actuellement : des tables , des rôles, des modeles de transaction.

La fonctionnalité que nous pouvons démontrer maintenant est : Tout le site

Ce qui nous reste à faire en priorité est : s'assurer le bon fonctionnement du parcours à travers le site.

13. Feedback de l'enseignant

Critère	Feedback
Clarté de l'idée	A REMPLIR
Avancement technique	A REMPLIR
Qualité du modèle de données	A REMPLIR
Respect du MVP	A REMPLIR
Organisation de l'équipe	A REMPLIR
Prochaine action recommandée	A REMPLIR

14. Message final

Un hackathon ne se gagne pas avec la quantité d'idées. Il se gagne avec la capacité à choisir, construire, tester et présenter.

Un petit parcours complet qui fonctionne vaut mieux qu'une grande application incomplète.

Priorisez le scénario principal :

Organisateur crée un tournoi → crée une équipe → joueur demande à rejoindre → organisateur accepte → joueur rejoint l'équipe

Si ce scénario fonctionne, votre projet possède déjà un coeur solide.

Clarifiez. Priorisez. Codez. Testez.

Bon courage pour la suite du hackathon.